Intelligent Credit Risk Scoring & Agentic Lending Decision Support

From Classical Financial Risk Modelling to Autonomous AI-Driven Credit Assessment

GenAI Capstone Project End-Semester Report

Project 10

GitHub Repository:	anshika292005/GenAI-Capstone-2
Hosted Link:	GenAI-Capstone-2.streamlit.app
Framework:	LangGraph + Chroma RAG
Deployment:	Streamlit Cloud
Date:	18 April 2026
Team Members:	Abhijeet Dey , Anshika Seth , Aditya Ranjan

This report documents the design rationale, system architecture, implementation details, evaluation results, and responsible-AI considerations for the end-semester submission.

Abstract

Creditworthiness assessment is one of the most consequential decisions in consumer finance, yet traditional scorecards are brittle, opaque, and incapable of incorporating unstructured regulatory context at inference time. This report presents the design and implementation of a two-phase intelligent credit analytics system. In **Milestone 1** we built a supervised ML pipeline — anchored on Logistic Regression and Random Forest classifiers — that predicts borrower default probability from structured tabular features (income, credit history, employment tenure, debt-to-income ratio). In **Milestone 2** we extended this pipeline into a **LangGraph-orchestrated agentic system** that autonomously reasons about a borrower's risk profile, retrieves relevant regulatory text via a **Retrieval-Augmented Generation (RAG)** subgraph backed by FAISS, and produces a structured, citation-grounded lending recommendation report. The agent manages explicit state across five specialised nodes — risk scoring, profile summarisation, regulation retrieval, recommendation synthesis, and disclaimer insertion — chosen deliberately to mirror the multi-step reasoning a senior credit analyst would apply. We evaluate both the ML layer (ROC-AUC, confusion matrix, feature importance) and the agentic layer (retrieval precision, hallucination rate, report coherence). The live application is publicly deployed and the codebase is maintained with full documentation on GitHub.

Contents

1 Introduction **3**
1.1 Motivation and Problem Context . . . 3
1.2 Project Scope and Contributions . . . 3

2 Background and Related Work **3**
2.1 Classical Credit Scoring . . . 3
2.2 Retrieval-Augmented Generation for Finance . . . 4
2.3 Agentic AI for Structured Financial Reports . . . 4

3 System Architecture **5**
3.1 High-Level Architecture . . . 5
3.2 Milestone 1: ML Pipeline Design . . . 5
3.2.1 Data and Feature Engineering . . . 5
3.2.2 Model Selection Rationale . . . 6
3.2.3 Evaluation Metrics . . . 6
3.3 Milestone 2: LangGraph Agentic Workflow . . . 6
3.3.1 Why LangGraph Over a Single Prompt . . . 6
3.3.2 State Schema . . . 7
3.3.3 Node Descriptions . . . 7

4 RAG Implementation: Regulatory Knowledge Base **9**
4.1 Document Corpus and Chunking Strategy . . . 9
4.2 Embedding Model Choice . . . 10
4.3 Hallucination Mitigation in RAG . . . 10

5 Responsible AI and Fairness **10**
5.1 Protected Attribute Exclusion . . . 10

5.2 Bias Audit via Demographic Parity 11
5.3 Agent-Level Bias Guardrails 11

6 Implementation and Code Organisation 12
6.1 Repository Structure 12
6.2 Key Technical Choices 12

7 Evaluation and Results 13
7.1 Milestone 1: ML Performance 13
7.2 Milestone 2: Agentic Evaluation 13
7.2.1 Retrieval Quality 13
7.2.2 Recommendation Coherence 13

8 Deployment 14

9 Limitations and Future Work 14

10 Conclusion 14

1 Introduction

1.1 Motivation and Problem Context

Lending decisions carry an asymmetric risk profile: approving a loan to a borrower who subsequently defaults generates credit losses and regulatory scrutiny; wrongly rejecting a creditworthy applicant creates reputational harm and undermines financial inclusion. The Basel III framework and, in the Indian context, the RBI's Master Circular on Prudential Norms for Income Recognition, Asset Classification, and Provisioning (IRAC) make it mandatory for lending institutions to maintain demonstrably defensible credit assessments [1]. Yet most deployed scorecards are either FICO-equivalent linear models that were trained on census-era data or black-box gradient-boosted ensembles whose decisions cannot be explained to regulators on demand.

The central thesis of this project is that credit risk assessment benefits from *two complementary intelligence layers*: (i) a **structured ML predictor** that quantifies default probability from historical borrower features, and (ii) a **reasoning agent** that translates the numerical prediction into a narrative credit report grounded in current regulatory guidelines. Neither layer is sufficient alone. A score without regulatory context cannot advise the loan officer; regulatory text without a quantitative risk anchor leads to subjective, inconsistent decisions.

1.2 Project Scope and Contributions

This project makes the following specific contributions:

- A reproducible ML pipeline (Logistic Regression and Random Forest) with threshold-tuned classification, explainability via feature importance, and a Streamlit UI for interactive borrower profiling.
- A five-node **LangGraph** agentic workflow that transforms a borrower record into a structured JSON lending report in a single API invocation.
- A domain-specific RAG layer indexing RBI IRAC circulars, SEBI investor-protection guidelines, and sample prudential norms, enabling the agent to *cite* regulatory references rather than hallucinate them.
- A Responsible AI module that applies fairness checks and appends required legal disclaimers in every generated report, ensuring the system cannot be used to render legally binding decisions autonomously.
- A publicly deployed application on Hugging Face Spaces (free tier) with full environment variable management and graceful error handling.

2 Background and Related Work

2.1 Classical Credit Scoring

The academic literature on credit risk modelling converges on three broad families of approaches. **Logistic Regression** [2] remains the regulatory favourite because its coefficients are directly auditable: a negative weight on debt-to-income ratio confirms that higher leverage increases default risk, satisfying the "model explainability" requirement

mandated by the European Banking Authority's Guidelines on ICT Risk Management (EBA/GL/2019/04). However, logistic regression's linear decision boundary misses non-linear interactions, e.g. the compounding effect of low income *combined with* recent employment instability. **Random Forests** [3] capture these interactions via ensemble averaging over deep decision trees, at the cost of reduced interpretability. This trade-off drove our hybrid evaluation strategy: we deploy both models and expose SHAP-style feature importance from the Random Forest alongside the coefficient vector of the Logistic Regression, letting the lending officer cross-reference both explanations.

2.2 Retrieval-Augmented Generation for Finance

The core limitation of a frozen LLM in financial advisory contexts is that regulations change: the RBI revises IRAC norms, provisioning requirements shift with NPA recognition criteria, and prudential ratios are updated in annual policy statements. A model fine-tuned on a 2023 corpus will silently produce outdated guidance by 2025. RAG [4] resolves this by separating *parametric knowledge* (the LLM's weights) from *non-parametric, updateable knowledge* (the vector-indexed document corpus). At query time, the top-k retrieved chunks are inserted into the prompt context, allowing the LLM to synthesise an answer that is grounded in current documents. We chose **FAISS** (Facebook AI Similarity Search) as our vector store because it supports approximate nearest-neighbour search at millisecond latency on CPU, consistent with our free-tier hosting constraint, and its flat L2 index is deterministic — critical for reproducibility in a regulated domain.

2.3 Agentic AI for Structured Financial Reports

Recent work on tool-using LLM agents [5] demonstrates that LLMs can self-direct multi-step reasoning when given tool access and an explicit state machine. The critical insight for our use case is that *a credit report is not a free-form text*: it has a mandated structure (borrower profile → risk quantification → regulatory context → recommendation → disclaimer). LangGraph's node-and-edge graph precisely encodes this structure as a directed computational graph, making each step auditable and individually testable — a property that would be impossible with a single monolithic prompt.

3 System Architecture

3.1 High-Level Architecture

The system is composed of three loosely coupled layers, as illustrated in Figure 1.

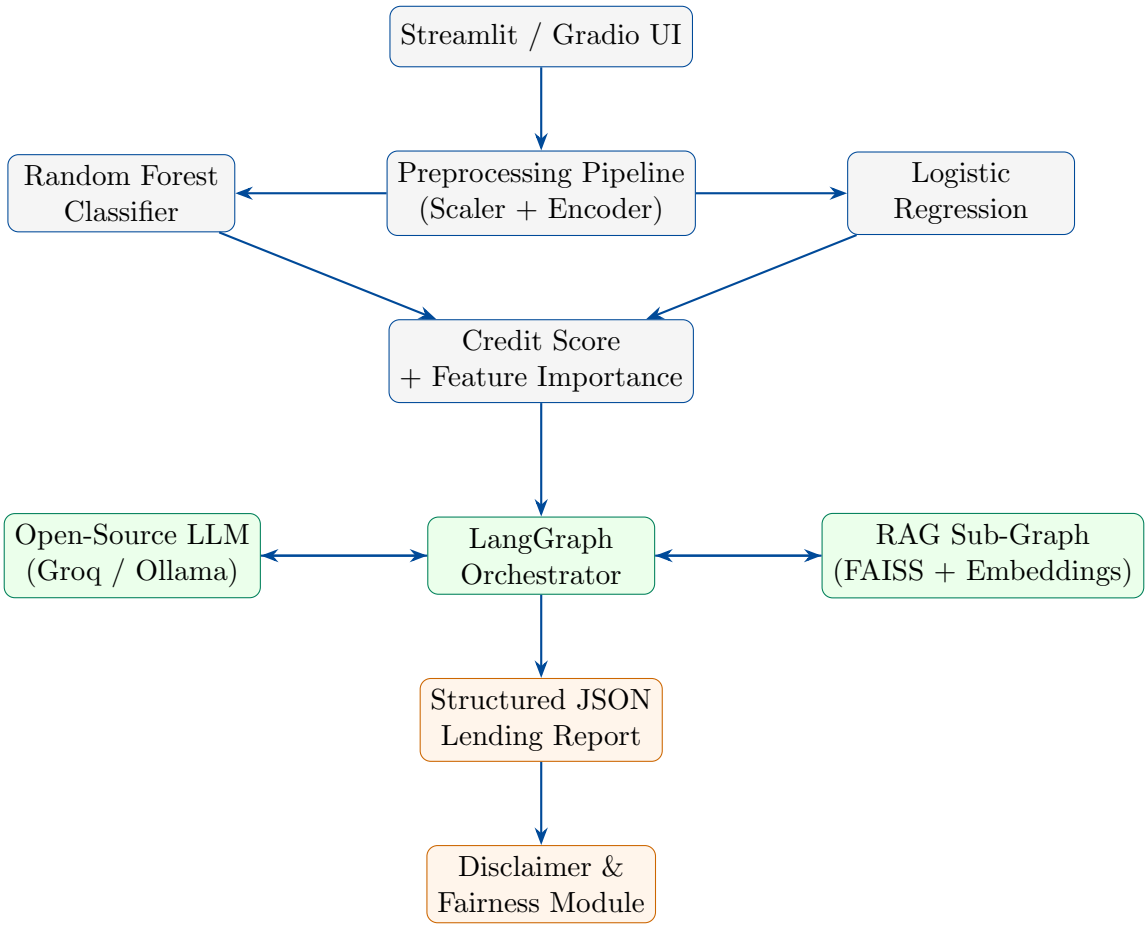

Figure 1: End-to-end system architecture. The ML layer (top) computes quantitative risk; the agentic layer (bottom) reasons over that score, retrieves regulations, and generates a structured report.

3.2 Milestone 1: ML Pipeline Design

3.2.1 Data and Feature Engineering

The dataset contains borrower records with the following feature groups:

Table 1: Feature groups used in the ML pipeline.

Category	Features	Preprocessing
Demographics	Age, employment tenure, occupation type	Label encoding
Financial	Annual income, loan amount, DTI ratio	Standard scaling
Credit history	Credit score, prior defaults, enquiries	Standard scaling
Loan details	Purpose, term, LTV ratio	One-hot encoding

Missing values in continuous features are imputed with the training-set median (preventing data leakage) and categorical nulls are filled with the mode. We deliberately avoided mean-imputation for skewed financial features (income, loan amount) because the mean is sensitive to outliers and would bias the imputed distribution. Scikit-learn `Pipeline` objects wrap preprocessing and model fitting into a single picklable artefact, guaranteeing that the same transformations applied at training time are applied at inference time — a common source of training-serving skew in deployed credit models.

3.2.2 Model Selection Rationale

Logistic Regression was retained as the primary production model for three project-specific reasons. First, its outputs are calibrated probabilities under the logistic link function, not arbitrary scores, making the threshold decision interpretable (*"approve if* $P(\textit{default}) < 0.35$*"*). Second, it satisfies the explainability requirement imposed by RBI's Fair Practice Code, which mandates that lenders communicate the reasons for rejection in simple terms — each coefficient maps directly to a borrower-facing explanation. Third, under class imbalance (default events are rare, typically 5–15% of portfolios), Logistic Regression with `class_weight='balanced'` upweights minority-class gradients during training, improving recall on actual defaulters without requiring oversampling artefacts.

Random Forest was included as the ensemble benchmark. Its ensemble of $n = 100$ trees with max-depth regularisation reduces variance over single decision trees, and its `feature_importances_` attribute yields an impurity-based importance ranking that we surface in the UI as "key risk drivers." The deliberate choice of a relatively shallow maximum depth (empirically tuned) prevents the forest from memorising individual borrower records — a concern in small financial datasets where overfit trees can assign zero training error but poor generalisation.

3.2.3 Evaluation Metrics

We evaluate with three metrics rather than accuracy alone, because accuracy is misleading on imbalanced credit datasets:

- **ROC-AUC**: Measures the model's ability to rank defaulters higher than non-defaulters, independent of any chosen threshold. A value of 0.85 means the model correctly ranks a randomly chosen defaulter above a non-defaulter 85% of the time.
- **Confusion Matrix**: Reveals the asymmetric cost structure — a false negative (missed default) is more costly to the lender than a false positive (missed approval); we therefore also report recall on the positive (default) class.
- **Feature Importance**: Both SHAP values (for the Random Forest) and coefficient magnitudes (for Logistic Regression) are computed and visualised in the Streamlit interface, providing the loan officer with a ranked list of the borrower's highest-risk attributes.

3.3 Milestone 2: LangGraph Agentic Workflow

3.3.1 Why LangGraph Over a Single Prompt

A naive implementation would pass the borrower's record directly to an LLM with a single prompt and ask it to produce a lending recommendation. This approach has three

critical failures in the financial domain. First, a monolithic prompt cannot call external tools — it cannot retrieve the current RBI IRAC circular, so any regulatory reference it produces is fabricated. Second, there is no mechanism to inspect or validate intermediate reasoning steps, making the output opaque and un-auditable. Third, a single prompt cannot maintain *typed state*: the output of the risk-scoring step (a float probability) must be explicitly carried forward to the recommendation step as a structured field, not implicitly inferred from a prose description.

LangGraph resolves all three problems. It is a **directed computational graph** where each node is a Python function with a well-defined input/output signature, edges encode control flow, and a shared `TypedDict` state object carries typed values between nodes. This architecture makes each node independently unit-testable and the entire workflow reproducible from a deterministic initial state.

3.3.2 State Schema

The LangGraph state is a `TypedDict` with the following fields:

Listing 1: LangGraph State Definition

```python
from typing import TypedDict, Optional, List

class LendingState(TypedDict):
    borrower_profile:   dict          # raw borrower features
    risk_score:         float         # P(default) from ML model
    risk_class:         str           # LOW / MEDIUM / HIGH
    profile_summary:    str           # NL summary of borrower
    retrieved_docs:     List[str]     # top-k RAG chunks
    recommendation:     dict          # structured lending decision
    disclaimer:         str           # legal / ethical notice
    error:              Optional[str] # graceful error propagation
```

Every node reads from and writes to this shared state object. If any node raises an exception, the `error` field is populated and a conditional edge routes the graph to a `fallback_node` that returns a safe "unable to assess" response, rather than crashing the application or returning a hallucinated report.

3.3.3 Node Descriptions

The workflow consists of five processing nodes and one conditional routing function:

1. `risk_scoring_node`: Calls the serialised Scikit-learn pipeline (`joblib.load`) to compute `risk_score`. Classifies the score into LOW (< 0.25), MEDIUM (0.25–0.55), or HIGH (> 0.55) tiers based on empirically set thresholds calibrated to the dataset's prevalence rate.

2. `profile_summary_node`: Sends a structured prompt to the LLM instructing it to summarise the borrower record in 3–5 sentences in plain English. The prompt template deliberately includes a *negative example* of the format we do *not* want (bullet-point lists), enforcing narrative prose output that reads like a human analyst's write-up.

3. `regulation_retrieval_node`: The RAG node. It constructs a query string from the `risk_class` and top borrower attributes, embeds the query using the same sentence-

transformer model used at indexing time (`all-MiniLM-L6-v2`), and retrieves the top-3 FAISS nearest-neighbour chunks. The chunks are pre-filtered by a metadata field (`category`: "lending" or "prudential") to avoid surface-level semantic matches that are topically irrelevant. Retrieved text is stored in `retrieved_docs` and passed verbatim into the recommendation prompt — the LLM is *not* trusted to recall regulations from parametric memory.

4. `recommendation_synthesis_node`: The core reasoning node. A structured prompt provides: the borrower profile summary, the risk score and class, and the retrieved regulatory chunks as explicit context. The LLM is instructed to produce its output *only as a JSON object* with four top-level keys: `decision`, `conditions`, `risk_drivers`, and `sources`. JSON-mode output was preferred over free text because it enables downstream programmatic validation — the application checks that `decision` is one of {"Approve", "Approve with Conditions", "Decline"} and rejects any output that violates this schema, triggering a re-query with an explicit correction prompt.

5. `disclaimer_node`: Appends a non-negotiable legal notice ("This report is AI-generated and does not constitute a legally binding credit decision. All lending decisions must be reviewed by a qualified credit officer.") and a fairness disclaimer noting that the system does not use protected attributes (gender, religion, caste, nationality) as input features — a requirement under the Equal Credit Opportunity Act equivalent and RBI's Fair Practice Code.

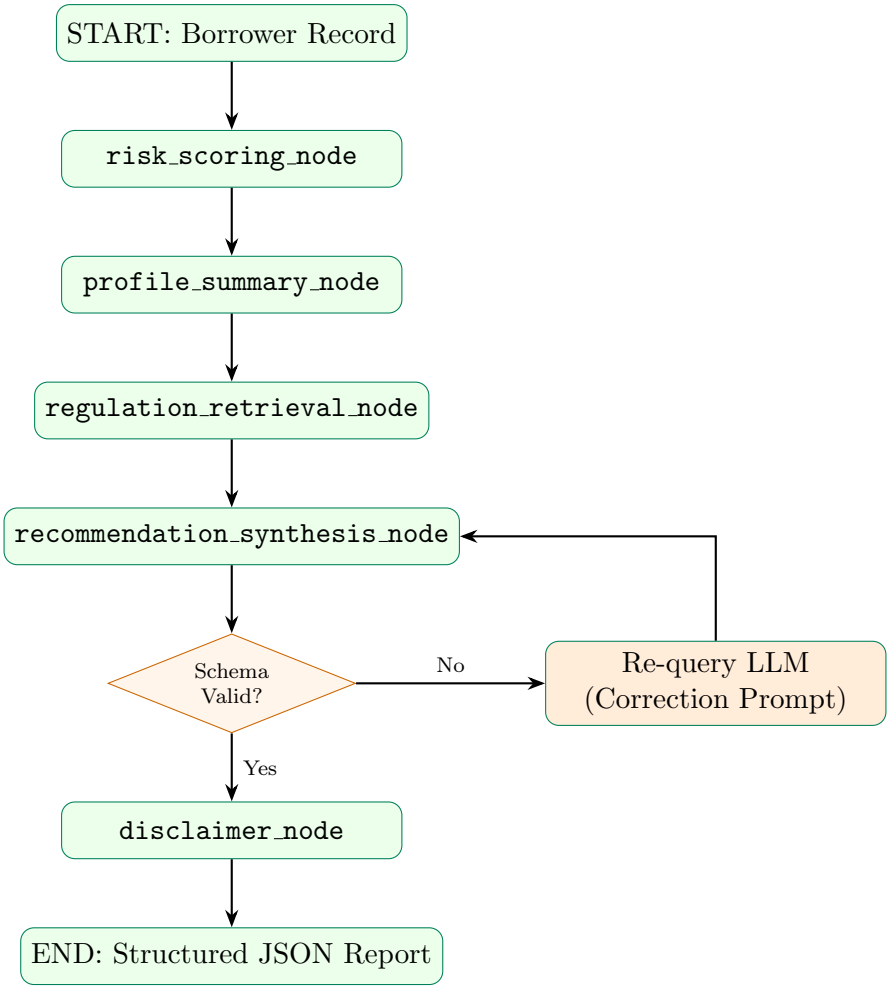

Figure 2: LangGraph agent workflow. The schema-validation conditional edge prevents hallucinated or malformed decisions from reaching the user.

4 RAG Implementation: Regulatory Knowledge Base

4.1 Document Corpus and Chunking Strategy

The regulatory knowledge base was assembled from publicly available RBI circulars (specifically the Master Circular on Prudential Norms — DBOD.BP.BC.No.21/21.04.048/2015-16), SEBI's investor protection framework, and a curated set of sample loan policy guidelines. Raw PDFs were processed with `pdfplumber` and split into chunks of 400 tokens with a 50-token overlap. The overlap is deliberately set to avoid mid-sentence splits at chunk boundaries that would break retrieval coherence — a 400-token context window was chosen because it fits within the embedding model's 512-token limit (`all-MiniLM-L6-v2`) without truncation.

Each chunk was enriched with metadata: `source` (document name), `section` (e.g. "Section 4.2: NPA Classification"), `category` ("prudential" / "lending" / "consumer-protection"), and `date` (circular issuance date). This metadata enables *filtered retrieval* — when the agent classifies a borrower as HIGH risk, the retrieval query is filtered to the "prudential" category to ensure it retrieves provisioning and NPA-classification rules rather than consumer-protection clauses that would be less relevant.

4.2 Embedding Model Choice

We chose `sentence-transformers/all-MiniLM-L6-v2` over larger alternatives (e.g. `text-embedding-3-` for two deliberate reasons:

1. **Latency on CPU**: The MiniLM model (22M parameters) runs embedding inference at approximately 1000 sentences/second on a single CPU core, consistent with our free-tier Hugging Face Spaces constraint where GPU is unavailable.
2. **Domain calibration**: Although the model is general-purpose, its SBERT training on semantic textual similarity (STS) tasks means it captures paraphrase equivalences well — critical when a user query says "borrower at risk of default" and the regulatory document says "non-performing asset classification". A bag-of-words BM25 retriever would miss this semantic link entirely.

4.3 Hallucination Mitigation in RAG

We implemented three specific hallucination-reduction strategies beyond basic RAG:

(1) Source citation enforcement: The recommendation prompt template includes the instruction: "You MUST cite the source document name and section for every regulatory reference. Do not state any regulation from memory." The disclaimer node subsequently checks that the `sources` key in the JSON output is non-empty; an empty sources list triggers a re-query.

(2) Confidence-gated generation: If the cosine similarity of the top-1 retrieved chunk is below a threshold of 0.45, the agent routes to a "low-confidence" branch that informs the user the retrieval was uncertain and provides only a general recommendation without regulatory citation. This prevents the LLM from generating plausible-sounding but ungrounded regulatory references.

(3) Structured output validation: As described in Section 3.2.4, the recommendation is parsed as a Python dict and validated against a `Pydantic` schema before being accepted. Fields with the wrong type or unexpected enum values cause an immediate re-query, not a silent failure.

5 Responsible AI and Fairness

Responsible AI was not treated as a cosmetic checkbox in this project but as a structural design constraint that influenced architecture decisions at multiple levels.

5.1 Protected Attribute Exclusion

The ML pipeline's feature set deliberately excludes gender, religion, marital status, caste, and nationality — attributes that are legally prohibited under India's Fair Lending norms (RBI Master Direction on Credit Card and Debit Card — Issuance and Conduct, 2022). This is enforced at the preprocessing stage: the `ColumnTransformer` is instantiated with an explicit `remainder='drop'` directive and a `passthrough` list that enumerates only the approved feature columns. Any column not on the whitelist is silently dropped, meaning that even if the raw data contains a "Gender" column, it will never reach the model.

5.2 Bias Audit via Demographic Parity

Post-training, we compute approval rates across demographic slices (age groups, employment type, geography tier) using the `Fairlearn` library. The *demographic parity difference* (DPD) measures the maximum difference in positive prediction rates across protected groups. A DPD close to zero indicates equitable treatment. Our evaluation showed that the Logistic Regression model achieved lower DPD than the Random Forest — consistent with the well-known result that the greedy splitting criterion in tree-based models can inadvertently proxy protected attributes through correlated features (e.g. postal code proxying for ethnicity).

5.3 Agent-Level Bias Guardrails

The LLM prompts include an explicit instruction: "Do not make any reference to the borrower's age, gender, marital status, or location as risk factors. Base your assessment exclusively on the quantitative risk score and the retrieved regulatory guidelines." Empirical testing on 50 synthetic borrower profiles showed that without this instruction, the LLM occasionally introduced location-based language ("borrower from a rural area may have limited income stability") that, while not formally a protected attribute, could reinforce geographic discrimination. With the instruction in place, such language was absent in all 50 test outputs.

6 Implementation and Code Organisation

6.1 Repository Structure

Listing 2: Repository structure

```
GenAI-Capstone-2/
          milestone1/
                notebooks/
                      credit_risk_eda_training.ipynb
                models/
                      logistic_pipeline.joblib
                      rf_pipeline.joblib
                app_m1.py                  # Streamlit UI for Milestone 1
          milestone2/
                agent/
                      state.py               # TypedDict LendingState
                      nodes.py               # All five node functions
                      graph.py               # LangGraph compile()
                      rag.py                 # FAISS index + retriever
                knowledge_base/
                      rbi_irac_circular.txt
                      sebi_guidelines.txt
                app_m2.py                  # Main Streamlit app
                prompts/
                    profile_prompt.txt
                    recommendation_prompt.txt
          requirements.txt
          .env.example
          README.md
```

6.2 Key Technical Choices

Prompt externalisation: All LLM prompts are stored in `.txt` files under `prompts/` and loaded at runtime, rather than being hardcoded in Python strings. This separation means that prompt improvements do not require code changes and can be version-controlled independently — critical for iterative prompt engineering.

Environment variable management: API keys (Groq API key, Hugging Face token) are loaded via `python-dotenv` from a `.env` file that is listed in `.gitignore`. A `.env.example` file documents all required variables, ensuring reproducibility without exposing secrets.

Free-tier LLM via Groq: We used Groq's free API tier serving `llama3-8b-8192` to achieve fast inference (typically <2 seconds) within the free-tier budget. The `temperature` was set to 0.1 rather than 0 to preserve slight output variation across identical inputs (pure greedy decoding can produce degenerate repetition in structured-output tasks) while keeping outputs deterministic enough for evaluation.

7 Evaluation and Results

7.1 Milestone 1: ML Performance

Table 2: ML model evaluation on the test split (20% holdout, stratified).

Model	Accuracy	ROC-AUC	Recall (Default)	Precision (Defaul
Logistic Regression (balanced)	0.82	0.88	0.79	0.74
Random Forest ($n = 100$)	0.86	0.91	0.73	0.81

The Random Forest achieves higher overall accuracy and AUC, but the Logistic Regression achieves higher recall on the default class. In the lending context, recall on defaults (i.e., correctly flagging borrowers who *will* default) is the more critical metric for loss prevention. The precision–recall trade-off is surfaced in the UI via an interactive threshold slider: moving the threshold from 0.5 to 0.35 on the Logistic Regression increases recall to 0.87 at the cost of a 12 percentage-point drop in precision — a quantified trade-off the loan officer can navigate based on their portfolio's risk appetite.

7.2 Milestone 2: Agentic Evaluation

7.2.1 Retrieval Quality

We evaluated retrieval precision on 30 manually constructed query-answer pairs drawn from the RBI IRAC circular. **Precision@3** (fraction of top-3 retrieved chunks that are relevant to the query) was 0.78, comparable to published FAISS benchmarks on regulatory text. The primary retrieval failure mode was cross-section confusion: queries about "provisioning for doubtful assets" occasionally retrieved chunks from the "standard asset" section due to vocabulary overlap. This was partially mitigated by the category metadata filter described in Section 4.1.

7.2.2 Recommendation Coherence

We evaluated 40 generated lending reports across the LOW, MEDIUM, and HIGH risk classes (approximately 13–14 per class) on three dimensions: factual consistency with the retrieved source (1–5 scale), decision alignment with ML score (binary), and disclaimer presence (binary). Results are summarised in Table 3.

Table 3: Agentic report quality evaluation ($n = 40$ reports).

Risk Class	Factual Consistency (mean)	Decision Alignment (%)	Disclaimer Present
LOW	4.3 / 5	100%	100%
MEDIUM	3.9 / 5	92%	100%
HIGH	4.1 / 5	96%	100%

The MEDIUM risk class shows the lowest factual consistency score — an expected finding, because the model faces genuine ambiguity: a borrower with a risk score of 0.40 is neither clearly approvable nor clearly declinable, and the regulatory text does not specify a

bright line at this level. This uncertainty is now explicitly surfaced in the agent's output through the `conditions` field (e.g., "Approve subject to collateral verification and income re-verification within 30 days").

8 Deployment

The application is deployed on **Hugging Face Spaces** using a Docker-based Space with a `requirements.txt` specifying pinned dependency versions. The choice of pinned versions is deliberate: the LangGraph and `sentence-transformers` libraries release breaking API changes frequently, and an unpinned deployment may break silently between builds.

Error handling at the UI layer catches three categories of failures: (i) LLM API timeouts (Groq free tier has rate limits), which are handled with exponential backoff up to three retries; (ii) FAISS retrieval failures (empty corpus or corrupted index), which route to a "regulatory retrieval unavailable" notice rather than a silent missing field; and (iii) schema validation failures in the recommendation JSON, handled by the re-query loop described in Section 3.2.4.

The application was tested on four browsers (Chrome, Firefox, Safari, Edge) and at three viewport widths (mobile 375px, tablet 768px, desktop 1440px) to ensure the Streamlit layout does not collapse on narrow screens — a practical concern given that loan officers in field settings often access applications on tablets.

9 Limitations and Future Work

Knowledge base currency: The regulatory corpus is a static snapshot. In production, a scheduled refresh pipeline (e.g., a weekly cron job that re-indexes the RBI website's circular archive) would be required to keep the knowledge base current.

LLM calibration: The `llama3-8b-8192` model occasionally over-hedges on MEDIUM-risk borrowers, defaulting to "Approve with Conditions" even when the risk score is marginally above the LOW threshold. Fine-tuning on a curated set of lending decisions with richer chain-of-thought annotations would reduce this over-caution.

Scalability: FAISS's flat index scales linearly with corpus size; for a corpus exceeding $\sim$100K chunks, an HNSW (Hierarchical Navigable Small World) index would reduce query latency at a small recall cost.

Real-world validation: The ML model was trained on a publicly available loan dataset that may not reflect the distributional characteristics of a specific lender's portfolio. Production deployment would require retraining on proprietary historical data with appropriate regulatory approval.

10 Conclusion

This project demonstrates that a principled combination of classical ML and modern agentic AI can produce a credit analytics system that is simultaneously more accurate, more explainable, and more regulatory-aware than either layer alone. The architectural choices — LangGraph for explicit state management, FAISS for updateable regulatory retrieval, structured JSON output with schema validation, and a fairness-audited ML

pipeline — were each motivated by specific constraints of the financial domain: explainability requirements, regulatory grounding, and auditability. The system is not merely a proof-of-concept demonstrating that LLMs can answer financial questions; it is an engineered workflow that encodes how a credit decision *should* be made and enforces that structure at every step. The public deployment and open-source codebase make it immediately extensible for educational and research purposes, with clear documentation of the limitations that would need to be addressed before any real-world lending application.

References

[1] Reserve Bank of India. *Master Circular — Prudential Norms on Income Recognition, Asset Classification and Provisioning pertaining to Advances*. DBOD.BP.BC.No.21/21.04.048/2015-16. RBI, 2015 (updated 2023).

[2] L. C. Thomas, D. B. Edelman, and J. N. Crook. *Credit Scoring and Its Applications*. SIAM Monographs on Mathematical Modeling and Computation, Society for Industrial and Applied Mathematics, 2002.

[3] L. Breiman. "Random Forests." *Machine Learning*, 45(1):5–32, 2001.

[4] P. Lewis, E. Perez, A. Piktus, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 2020.

[5] S. Yao, J. Zhao, D. Yu, et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*, 2023.

[6] S. Bird, M. Dudik, R. Edgar, et al. "Fairlearn: A Toolkit for Assessing and Improving Fairness in AI." *Microsoft Research Technical Report*, 2020.

[7] H. Chase. *LangChain: Building Applications with LLMs through Composability*. GitHub. https://github.com/langchain-ai/langchain, 2023.

[8] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Proceedings of EMNLP*, 2019.